

Automated Backup and Recovery System Project

Project: Automated Backup and Recovery System

Services: EC2, EBS, Lambda, CloudWatch Events

Description: Automate EBS snapshot creation and deletion using Lambda triggered by CloudWatch Events. Implements snapshot retention policies.

Skills: Automation, backup strategies, Lambda scripting.

Definition: The Automated Backup and Recovery System is designed to provide reliable and fully automated backups of AWS EC2 instance volumes (EBS). It uses AWS Lambda to create snapshots of volumes selected by specific tags and automatically deletes older snapshots based on a defined retention policy. CloudWatch Events schedules the Lambda function at regular intervals, enabling seamless, automatic backup processes.

Scope of the Project: The scope of this project is to design and implement a fully automated backup and recovery system for AWS EC2 instance volumes (EBS). The system leverages AWS Lambda and CloudWatch Events to automate the creation of snapshots and deletion of older snapshots based on a defined retention policy.

This project covers:

- **Automated EBS Snapshot Creation:** Automatically creates backups of all EBS volumes in the account.
- **Snapshot Retention Management:** Deletes older snapshots according to the retention policy to optimize storage and control costs.
- **Scheduled Automation:** Backups are triggered at regular intervals using CloudWatch Events, eliminating the need for manual intervention.

Methodology:

1.Lambda + CloudWatch Events - Best Method

- Uses all required AWS services: EC2, EBS, Lambda, and CloudWatch Events.
- Provides full control over backup workflows, including custom tagging and retention policies.
- Fully automated: snapshots are created and deleted without manual intervention.
- Production-ready and suitable for organizational deployment.
- Cost-effective, Free-tier friendly for small workloads
- Requires Lambda scripting and CloudWatch Event scheduling, but provides maximum flexibility and control.

2.EC2 + CRON + AWS CLI – Alternative Method

- Uses EC2 instances, EBS, CRON scheduler, and AWS CLI for backup automation.
- Enables full customization of backup logic, snapshot tagging, naming and retention policies.
- Automates backups using cron jobs and shell or Python scripts.

- Requires a continuously running EC2 instance, along with OS patching, monitoring, and security
- management.
- Involves higher operational effort and cost compared to serverless solutions.
- Best suited for legacy systems or use cases requiring persistent compute and deep customization.

AWS Services used in the Project:

1.Amazon EC2 (Elastic Compute Cloud)

- Definition: Amazon EC2 is a web service that provides resizable virtual servers in the cloud, allowing users to run applications and workloads on demand. Automated Backup and Recovery System
- Purpose: Provides virtual servers (instances) where applications run and EBS volumes are attached.
- Role: The root and attached EBS volumes of EC2 instances are the primary targets for snapshot creation.

2.Amazon EBS (Elastic Block Store)

- Definition: Amazon EBS is a managed block storage service designed to be used with EC2 instances, offering persistent and reliable storage.
- Purpose: Provides persistent block storage for EC2 instances.
- Role: Volumes are backed up by taking snapshots.

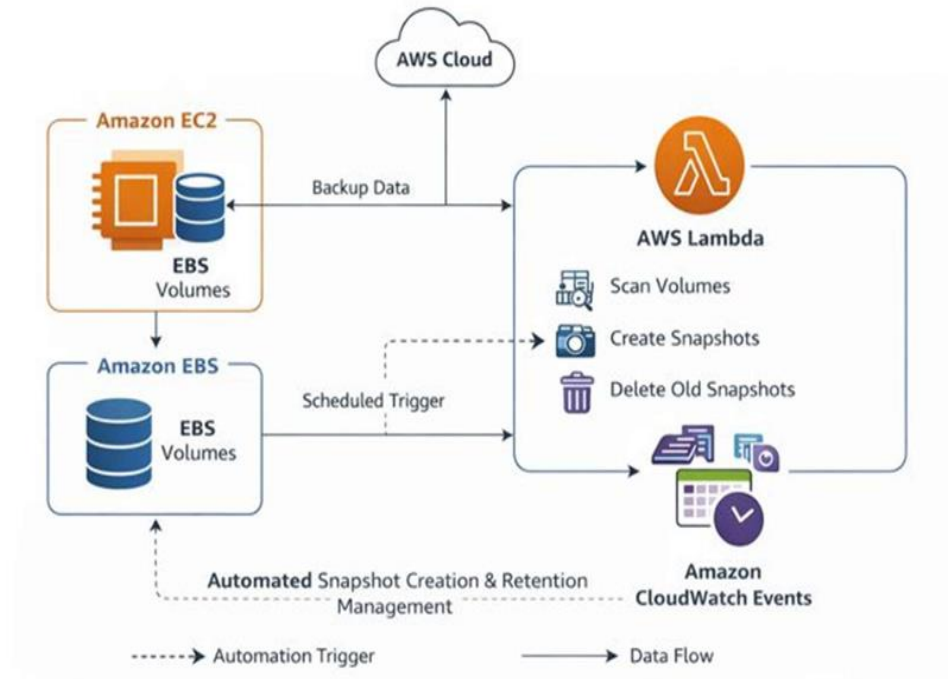
3.AWS Lambda

- Definition: AWS Lambda is a serverless compute service that allows users to run code without provisioning or managing servers.
- Purpose: Runs serverless code to automate backup operations.
- Role: Scans EC2 volumes, Creates snapshots of EBS volumes., Deletes old snapshots based on retention policy.

4.Amazon CloudWatch Events

- Definition: Amazon CloudWatch Events (now part of Amazon Event Bridge) is a service that delivers a near real-time stream of system events and enables automated actions based on schedules or events.
- Purpose: Schedules and triggers automated tasks.
- Role: Triggers the Lambda function at regular intervals to ensure snapshots are created and old snapshots deleted without manual intervention.

Architecture Diagram:



Implementation Steps to Do the Project:

Step1. Launch EC2 Instance

- Go to AWS Console → EC2 → Launch Instances.
- Choose Amazon Linux.
- Configure: Default VPC, Default Subnet, Auto-assign Public IP.
- Add Root Volume: 8 GB, gp2.
- Configure Security Group
- Review & Launch.

The screenshot shows the AWS Management Console interface for the EC2 service. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, and a list of EC2-related services. The main content area displays the 'Instances (1/1)' page. A table lists the instance 'backup-server' with ID 'i-01c002af23a826cb7', state 'Running', type 't3.micro', and availability zone 'ap-south-1a'. Below the table, the 'Details' tab is selected, showing the instance summary with the same ID, public IP address, and private IP addresses.

- Add Tag to volume: Key=Backup, Value=True.

This screenshot shows the same AWS Management Console interface as the previous one, but with the 'Tags' tab selected for the instance 'i-01c002af23a826cb7'. The 'Tags' section shows a single tag with the key 'Backup' and the value 'True'.

Step 2: Create IAM Role for Lambda

- Go to IAM → Roles → Create Role → Lambda.
- Attach policy: AmazonEC2FullAccess
- Name the role: Lambda-EBS-Backup-Role

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

Role Lambda-EBS-Backup-Role Info

Allows Lambda functions to call AWS services on your behalf.

Summary Edit

Creation date
May 22, 2026, 14:36 (UTC+05:30)

ARN
arn:aws:iam::973629900070:role/Lambda-EBS-Backup-Role

Last activity
16 minutes ago

Maximum session duration
1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (2) Info

You can attach up to 10 managed policies.

Simulate Remove Add permissions

Step 3: Create Lambda Function

- Go to Lambda → Create Function → Author from Scratch.
- Function Name: EBS-Auto-Backup
- Runtime: Python 3.10.
- Add Existing Role: Lambda-EBS-BackupRole.

Functions (1) Last fetched 5/22/2026, 7:16:13 PM

Search by attributes or search by keyword

Function name	Description	Package type	Runtime	Type	Last modified
EBS-Auto-Backup	-	Zip	Python 3.12	Standard	50 minutes ago

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Step 4: Lambda Function code and Test

- Add the backup script
- In Lambda console, click Test → Create Test Event.
- Enter Event Name as test1.
- Click Test to run the Lambda function.
- Go to EC2 → Snapshots to confirm snapshot creation.
- Verify tags and description on the snapshot.

The screenshot shows the AWS Lambda console for the 'EBS-Auto-Backup' function. The 'Code' tab is selected, displaying the 'lambda_function.py' file. The code is as follows:

```

8  def lambda_handler(event, context):
21     for reservation in instances['Reservations']:
22         for instance in reservation['Instances']:
33             description = f"Backup-{instance_id}-{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}"
34
35             snapshot = ec2.create_snapshot(
36                 VolumeId=volume_id,
37                 Description=description
38             )
39
40             snapshot_id = snapshot['SnapshotId']

```

On the right, the 'Tutorials' section is visible, featuring a tutorial titled 'Create a simple web app'.

The screenshot shows the AWS Lambda console for the 'EBS-Auto-Backup' function, with the 'Test' tab selected. It displays a successful test execution result.

Executing function: succeeded (logs [logs](#))

Details

```

{
  "statusCode": 200,
  "body": "Backup process completed"
}

```

Summary

Code SHA-256 /k56RctqQvdyITMHZtwXBvo6hZl8ft2M0Len40HsX/o=	Execution time 3 hours ago
Function version \$1 ATEFT	Request ID 7516d4a8-Rc56-406e-878c-d7h79044f7e1

On the right, the 'Tutorials' section is visible, featuring a tutorial titled 'Create a simple web app'.

Step 5: Schedule Lambda Using CloudWatch Events

- Go to Event Bridge (CloudWatch Events)
- Event Source: Select Schedule → Recurring Schedule
- Schedule Pattern: Choose Cron expression
- Enter: cron (0/10 * * ? *)
- This triggers the Lambda every 10 minutes.
- Target: Add Lambda Function → AutomatedEBSBackup
- Enable Schedule
- Create Schedule

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

Amazon EventBridge > Scheduled rules > Daily-EBS-Backup

Amazon EventBridge <

- Dashboard
- Developer resources
 - Learn
 - Sandbox
 - Quick starts
- Buses
 - Event buses
 - Rules [Updated](#)
 - Global endpoints
 - Archives
 - Replays
- Pipes
 - Pipes

Daily-EBS-Backup

[Edit](#) [Disable](#) [Delete](#) [CloudFormation Template](#)

Rule details [Info](#)

Rule name Daily-EBS-Backup	Status Enabled	Event bus name default	Type Scheduled Standard
Description -	Rule ARN arn:aws:events:ap-south-1:973629900070:rule/Daily-EBS-Backup	Event bus ARN arn:aws:events:ap-south-1:973629900070:event-bus/default	

[Event schedule](#) [Targets](#) [Monitoring](#) [Tags](#)

Event schedule [Info](#)

Fixed rate of
5 minute

[Edit](#)

Step 6: Verify Snapshots & Retention

- Go to EC2 → Snapshots
- Automatically snapshot of volume for every 5 minutes.
- Snapshots older than retention period are deleted automatically.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

EC2 <

- Dashboard
- AWS Global View
- Events
- Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager [New](#)
- Images

Snapshots (2) [Info](#)

Last updated less than a minute ago [Recycle Bin](#) [Actions](#) [Create snapshot](#)

Snapshot scope: Owned by me [Search](#)

☐	Name ✎	Snapshot ID	Full snapshot size	Volume size	Description	Storage
☐		snap-0b998002fd5ab4404	1.64 GiB	8 GiB	Backup-i-01c002af23a826...	Standard
☐		snap-06f29d9e5396d5ac2	1.64 GiB	8 GiB	Backup-i-01c002af23a826...	Standard

Select a snapshot above.

EC2

Dashboard

AWS Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

▼ Images

snap-06f29d9e5396d5ac2

Last updated less than a minute ago

Delete

Actions

Details

Snapshot ID

973629900070

Full snapshot size

1.64 GiB

Progress

100%

Snapshot status

Completed

Owner

973629900070

Started

Fri May 22 2026 18:26:09 GMT+0530 (India Standard Time)

Product codes

-

Fast snapshot restore

-

Description

Backup-i-01c002af23a826cb7-2026-05-22-12-56

▼ Source volume

Volume ID

vol-058b08fb538723475

Volume size

8 GiB

▼ Encryption



Search

[Alt+S]





1



Asia Pacific (Mumbai) ▼ Maheshwari (973629900070) ▼ Maheshwari

EC2

Dashboard

AWS Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

▼ Images

Snapshots (1)

Last updated less than a minute ago

Recycle Bin

Actions

Create snapshot

Snapshot scope

Owned by me

Search

1

Name

Snapshot ID

Full snapshot size

Volume size

Description

Storage

973629900070

1.64 GiB

8 GiB

Backup-i-01c002af23a826cb7-2026-05-22-12-56

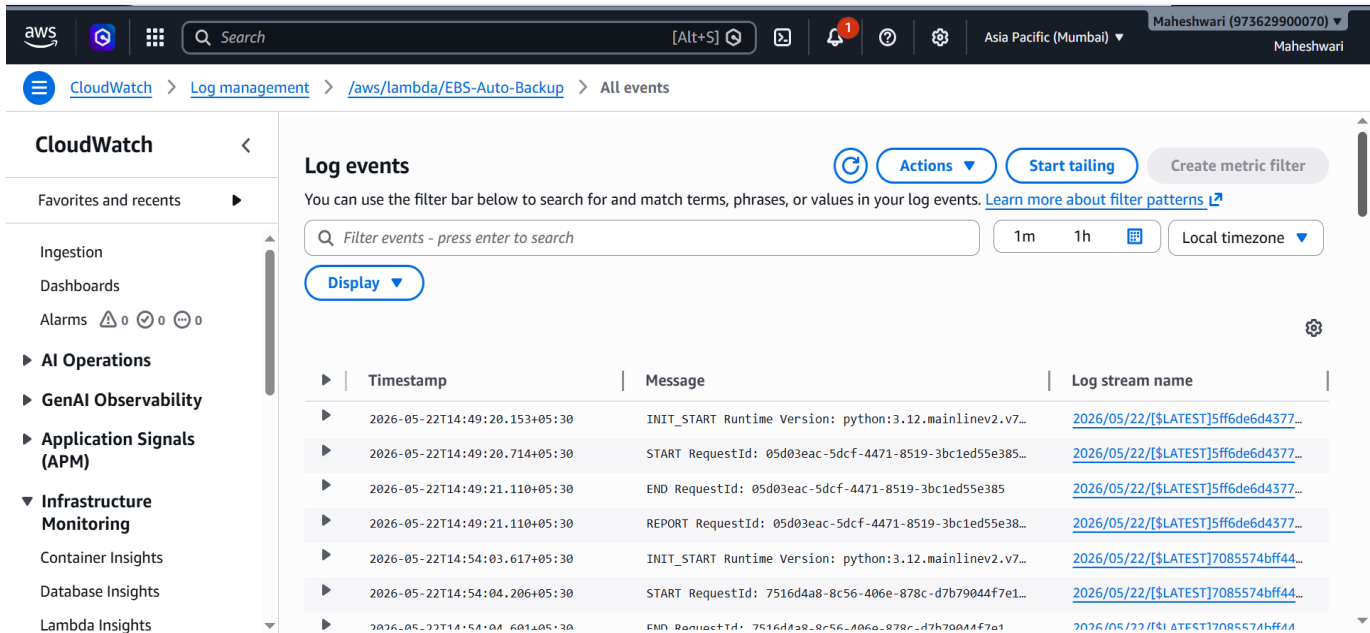
Standard

Select a snapshot above.

Step 7: CloudWatch Monitoring

CloudWatch logs were used to monitor:

- Lambda execution
- Snapshot creation
- Snapshot deletion



The screenshot shows the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, a search bar, and user information for Maheshwari (973629900070) in the Asia Pacific (Mumbai) region. The breadcrumb trail indicates the path: CloudWatch > Log management > /aws/lambda/EBS-Auto-Backup > All events.

The left sidebar contains the CloudWatch navigation menu with options like Favorites and recents, Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), and Infrastructure Monitoring. Under Infrastructure Monitoring, there are links for Container Insights, Database Insights, and Lambda Insights.

The main content area is titled 'Log events' and includes a search bar with the placeholder text 'Filter events - press enter to search'. Below the search bar is a 'Display' button. To the right of the search bar are filters for time range (1m, 1h) and a 'Local timezone' dropdown. A 'Start tailing' button is also present.

The log events are displayed in a table with the following columns: Timestamp, Message, and Log stream name. The table shows several log entries, including 'INIT_START Runtime Version: python:3.12.mainlinev2.v7...', 'START RequestId: 05d03eac-5dcf-4471-8519-3bc1ed55e385...', 'END RequestId: 05d03eac-5dcf-4471-8519-3bc1ed55e385...', and 'REPORT RequestId: 05d03eac-5dcf-4471-8519-3bc1ed55e385...'. Each entry has a corresponding log stream name link.

Troubleshooting:

Issue 1: Lambda Test Failed Due to Timeout.

The default Lambda timeout is 3 seconds, which is too short for the function to scan volumes, create snapshots, and

delete old snapshots.

Solution:

Increase the Lambda function timeout.

- Go to Lambda → Your Function → Configuration.
- Click Edit.
- Set Timeout to 5 minutes.
- Save the changes.

Issue 2: Lambda Handler Not Found

Lambda could not find the entry-point function because it was not defined.

Every Lambda function requires a handler function to start execution.

Solution:

Defined a Lambda handler function in code.

Conclusion:

The Automated Backup and Recovery System provide a reliable, automated solution for protecting EC2 data through scheduled EBS snapshots. Using Lambda and CloudWatch Events, backups are created and old snapshots are automatically deleted according to the retention policy, reducing manual effort and storage costs. The Lambda-based backup solution is scalable, flexible, and also can be applied across multiple instances or regions. It leverages AWS services efficiently, is suitable for organizational deployment, and can be implemented within the free-tier limits for small workloads.